PER SCHOLAS

Report

CAP 931: Capstone - Build a Sales Agent Prototype Using Multi-Agent GPT Models

Eduard Lukyanov

August 14th, 2026

Overview

This capstone developed a Sales Account Intelligence Agent, a Streamlit-based prototype that helps a sales representative turn public prospect website information into a structured account brief. The application collects product information, a prospect company website, a target buyer, and optional competitor URLs. It then retrieves public webpage content, extracts readable text, and prepares that context for account research and sales outreach generation.

The prototype was implemented with Python and managed with uv. The primary LLM path uses OpenAI gpt-4o-mini to produce a concise Markdown account brief. During testing, the OpenAI account returned an insufficient-quota error because API billing credits were exhausted. To keep the prototype usable and transparent, a clearly labeled rule-based fallback was added. The fallback is not presented as AI-generated; it formats the retrieved public webpage text and user inputs into the same account-brief structure.

Project goals

- Provide a simple Streamlit interface for entering product, prospect, and buyer information.
- Retrieve and parse public webpage content from a prospect URL.
- Use structured context and an LLM prompt to generate a research-based sales account brief.
- Handle API failure gracefully and preserve an end-to-end demo workflow through a labeled fallback output.

Steps

1. Project setup and dependency management

The project was initialized and managed with uv rather than raw pip. uv maintained the project configuration in pyproject.toml and locked resolved dependencies in uv.lock. This supports reproducible setup and avoids manually activating a virtual environment.

- Created the Python project and project files using uv.
- Installed Streamlit, requests, and beautifulsoup4 for the web interface and public-web retrieval.
- Installed openai and python-dotenv for LLM access and environment-variable loading.
- Ran the application with uv run streamlit run app.py.

Key dependencies: Streamlit for the web UI; requests for HTTP calls; BeautifulSoup for HTML parsing; OpenAI for the LLM client; python-dotenv for loading OPENAI_API_KEY from .env.

2. Streamlit interface

The Streamlit form captures the sales context needed to generate a targeted account brief. Required fields are validated before retrieval begins.

- Product name
- Product category
- Value proposition
- Prospect company website
- Target buyer
- Competitor URLs (optional)

The application normalizes URLs by adding https:// when needed and validates that the URL includes a scheme and network location before making a request.

3. Public-web data integration

The application retrieves the prospect webpage with requests and a browser-like User-Agent header. BeautifulSoup parses the returned HTML. Script, style, navigation, footer, header, and aside elements are removed before text extraction to reduce irrelevant content. The application stores the page title and truncates extracted public text to 5,000 characters for prompt context.

This process was successfully tested using Salesforce as a prospect URL. The application displayed a successful retrieval message, the source page title, and the source URL.

4. LLM capabilities

Model selection

The primary LLM path uses gpt-4o-mini. It was selected as an appropriate prototype model because it is designed to balance response quality, speed, and cost compared with larger GPT-family models. For an account-brief generation task, the model is intended to summarize supplied context, maintain a structured format, and draft outreach language. A limitation discovered during testing was API billing availability: the request returned an insufficient-quota error when the account had no remaining credits.

Prompt engineering

The prompt uses several basic prompt-engineering techniques: a defined role (B2B sales research assistant), explicit instructions not to invent facts, structured labels for every user input, public webpage title and extracted text, and an exact Markdown section outline. The prompt asks for Prospect Snapshot, Likely Business Focus, Sales Angle, Suggested Outreach Message, and Source Used. It also limits the outreach email to fewer than 120 words.

This is a first-pass structured prompt rather than an advanced multi-agent or chained workflow. Future iterations can compare prompts, separate research from drafting, and add source-specific verification steps.

Data integration

The prospect URL is used to retrieve public webpage text. That text, along with the page title and sales representative inputs, is passed to the LLM request. This grounds the intended LLM output in supplied public content instead of asking the model to generate a generic sales brief without source context.

5. Error handling and fallback

The app verifies that OPENAI_API_KEY is available before continuing. It also catches webpage retrieval errors and OpenAI request errors. When the OpenAI response indicates insufficient_quota or credit_balance_exhausted, the application shows a clear warning rather than a raw technical error.

A rule-based fallback brief is displayed only when live LLM generation cannot complete because of the quota issue. The fallback uses the page title, the first portion of retrieved public text, product details, value proposition, target buyer, and source URL. It is explicitly labeled as a fallback generated without live AI. This preserves transparency: the fallback is not represented as GPT output.

6. Testing evidence

Testing confirmed the following workflow:

1. The Streamlit application launched successfully at localhost:8501 using uv run streamlit run app.py.
2. The form accepted required product and prospect inputs.
3. The Salesforce webpage was retrieved successfully and the source information was displayed.
4. The OpenAI request path was reached but returned a 429 insufficient-quota error because the API account had no credits remaining.
5. The user-facing quota message displayed clearly.
6. The fallback account brief generated successfully after the quota condition was handled.

Conclusion

The Sales Account Intelligence Agent demonstrates a functional end-to-end prototype for collecting sales inputs, retrieving public prospect-webpage data, preparing an LLM-grounded account-brief request, and presenting a structured output. The project also demonstrates practical engineering decisions: dependency management with uv, environment-variable handling, input validation, public-web parsing, exception handling, and transparent degradation when an external API is unavailable.

The major limitation during testing was the exhausted OpenAI API billing quota. This was not a code failure; the client was able to reach the API request stage. The rule-based fallback ensured the application remained demonstrable, but it should be described accurately as a contingency output rather than an LLM-generated brief. Restoring API credits would allow the live gpt-4o-mini output path to be tested and evaluated directly.

Appendix

A. Commands used

```
uv add streamlit requests beautifulsoup4
uv add openai python-dotenv
uv tree
uv run streamlit run app.py
```

B. Evidence to attach

- Screenshot of the Streamlit form.
- Screenshot showing successful webpage retrieval and research source.
- Screenshot of the clean OpenAI billing-quota warning.
- Screenshot of the fallback generated account brief.
- Screenshot of uv tree showing installed dependencies.

- Screenshot of app.py, project file structure, and the uv run Streamlit command.

C. Future improvements

- Restore API credits and test live gpt-4o-mini generation with multiple prospect websites.
- Implement explicit competitor-page retrieval and competitor mention extraction.
- Add leadership-information extraction from reliable public sources or prospect pages.
- Add source citations per finding and link each insight to the page where it was found.
- Compare prompt versions or use a multi-step research-and-draft workflow.
- Add automated tests, rate limiting, caching, retries, and production deployment controls.